Университет Алатоо

AI Agents for Enterprise Processes Automation

ИИ-агенты для автоматизации корпоративных процессов

НЕДЕЛЯ 4 · АРХИТЕКТУРА АГЕНТА

Лабораторная работа 4

ReAct-агент с нуля: цикл «мысль → действие → наблюдение» на Python

Методические указания, задания, критерии оценивания и контрольные вопросы

Преподаватель: Эркебуланов Эрустан Усупбекович

2026–2027 учебный год

Данные о моделях и API актуальны на октябрь 2026 г.

Содержание

1. Цель и постановка задачи 3
2. Краткая теория: что мы строим 3
3. Подготовка окружения 3
   3.1. Структура проекта 4
4. Подготовка данных 4
5. Шаг 1. Инструменты 4
6. Шаг 2. Системный промпт 5
7. Шаг 3. Парсер и цикл агента 6
8. Шаг 4. Журнал шагов 7
9. Шаг 5. Тестирование и отладка 7
   9.1. Отладка без сети: подставная модель 7
   9.2. Тестовые вопросы 7
   9.3. Метрики по журналу 8
10. Задания 8
11. Отчёт и критерии оценивания 8
12. Типичные ошибки и отладка 9
13. Контрольные вопросы 9
14. Ответы на контрольные вопросы 11
15. Ссылки 12

1. Цель и постановка задачи

Цель: написать на чистом Python (без LangChain, LangGraph и других агентных фреймворков) ReAct-агента, который решает задачи в цикле «мысль → действие → наблюдение», использует два инструмента — калькулятор и поиск по текстовым документам — и записывает каждый шаг в журнал.

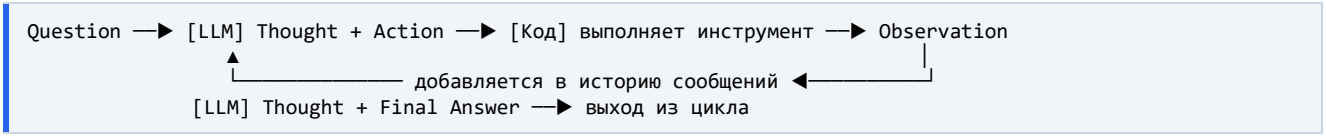
Сценарий: агент отвечает сотрудникам на вопросы по внутренним HR-документам компании, в которых нужно и найти факт, и что-то посчитать (суточные, отпускные дни, бонусы).

Время: 2 академических часа в аудитории + доработка и отчёт дома. **Сдача:** отчёт по шаблону из раздела 11 до следующего занятия (20% итоговой оценки составляют отчёты по лабораторным).

2. Краткая теория: что мы строим

ReAct (Reasoning + Acting) — паттерн, в котором языковая модель чередует рассуждение и действие (Yao et al., ICLR 2023) [1]. Агент работает в цикле: модель пишет мысль и выбирает инструмент, программный код выполняет инструмент, результат возвращается модели как наблюдение. Цикл повторяется, пока модель не напишет финальный ответ или не будет исчерпан лимит шагов.

Рисунок 1. Цикл ReAct, который реализуется в работе



Элемент	Кто пишет	Назначение
Thought	Модель	Рассуждение о следующем шаге; не влияет на внешний мир
Action + Action Input	Модель	Имя инструмента и входные данные — только намерение вызова
Observation	**Код**	Реальный результат инструмента или текст ошибки
Final Answer	Модель	Ответ пользователю; сигнал завершить цикл

> **Главная идея работы**
> Модель не выполняет инструменты сама — она только генерирует текст. Всё, что реально происходит (вычисления, поиск, проверки, логирование), делает ваш код. Поэтому именно код отвечает за безопасность и за остановку агента.

3. Подготовка окружения

Нужны Python 3.10+ и библиотека `openai`. Её используем не ради моделей OpenAI, а как универсальный клиент: Groq, Ollama и OpenRouter предоставляют OpenAI-совместимый API, поэтому достаточно поменять `base_url` и имя модели [8][10].

Листинг 1. Установка и проверка доступа

```
python -m venv .venv && source .venv/bin/activate   # Windows: .venv\Scripts\activate
pip install openai
export GROQ_API_KEY="gsk_..."                         # ключ из console.groq.com/keys

# Проверить, какие модели сейчас доступны вашему ключу:
curl https://api.groq.com/openai/v1/models -H "Authorization: Bearer $GROQ_API_KEY"
```

Актуально на октябрь 2026: какие модели выбрать

Список моделей Groq меняется. По странице документации, на момент подготовки материала в категории production доступны `openai/gpt-oss-20b` и `openai/gpt-oss-120b`, а `llama-3.3-70b-versatile` и `llama-3.1-8b-instant` переведены в Enterprise-доступ; для бесплатного и Developer-уровня Groq объявил их отключение с 16 августа 2026 [8][9]. Поэтому в лабораторной по умолчанию используется `openai/gpt-oss-20b`. Перед занятием проверьте список командой из листинга 1.

Альтернатива без интернета — Ollama на сервере курса или на ноутбуке: `base_url="http://localhost:11434/v1"`, `api_key="ollama"` (ключ обязателен для клиента, но не проверяется), модель — любая установленная instruct-модель (`ollama list`).

3.1. Структура проекта

Рекомендуемая структура: `agent.py` (промпт, парсер, цикл), `tools.py` (calculator, text_search), папка `docs/` с документами, `tests.py` с тестовыми вопросами и автоматически создаваемый журнал `agent_log.jsonl`.

Документы в `docs/` студенты пишут сами: 3–5 файлов по 5–10 абзацев, абзацы разделены пустой строкой. Включите в них числа (ставки, лимиты, сроки), чтобы для ответа требовалось и найти факт, и посчитать.

4. Подготовка данных

Агент отвечает на вопросы по «внутренним документам» вымышленной компании. Подготовьте 3–5 файлов `.txt` в папке `docs/`, по 5–10 абзацев в каждом; абзацы разделяйте пустой строкой — по ним работает поиск. Включите в тексты конкретные числа (ставки, лимиты, сроки), чтобы для части вопросов требовалось и найти факт, и вычислить результат.

Листинг 2. Пример файла docs/hr_policy.txt (данные вымышленные)

```
Ежегодный отпуск. Каждый сотрудник имеет право на ежегодный оплачиваемый отпуск
продолжительностью 28 календарных дней.

Командировки. Суточные при командировках по стране составляют 1500 сом в день.
Суточные выплачиваются за каждый день командировки, включая дни в пути.

Удалённая работа. Сотрудник может работать удалённо до 2 дней в неделю
по согласованию с руководителем.
```

Придумайте собственные правила — например, лимиты на обучение, ставки бонусов, нормы компенсации проезда. Чем разнообразнее документы, тем интереснее проверять поиск: в них должны встречаться похожие слова в разных контекстах.

5. Шаг 1. Инструменты

Калькулятор приводится полностью, потому что это вопрос безопасности, а не алгоритмики (см. раздел 6.6 учебного материала недели 4). Разберитесь, как он работает: выражение превращается в синтаксическое дерево, и вычисляются только разрешённые узлы.

Листинг 3. tools.py — безопасный калькулятор

```
import ast, operator

_OPS = {ast.Add: operator.add, ast.Sub: operator.sub, ast.Mult: operator.mul,
        ast.Div: operator.truediv, ast.Mod: operator.mod, ast.Pow: operator.pow,
        ast.USub: operator.neg, ast.UAdd: operator.pos}

def calculator(expression: str) -> str:
    """Безопасный калькулятор: только числа и + - * / % ** (без eval!)."""
    def _eval(node):
        if isinstance(node, ast.Constant) and isinstance(node.value, (int, float)):
            return node.value
        if isinstance(node, ast.BinOp) and type(node.op) in _OPS:
            left, right = _eval(node.left), _eval(node.right)
            if isinstance(node.op, ast.Pow) and abs(right) > 100:
```

```
                raise ValueError("слишком большая степень")   # защита от 9**9**9
            return _OPS[type(node.op)](left, right)
        if isinstance(node, ast.UnaryOp) and type(node.op) in _OPS:
            return _OPS[type(node.op)](_eval(node.operand))
        raise ValueError("допустимы только числа и операторы + - * / % **")
    try:
        expr = expression.strip().strip("`'\"").replace("^", "**")
        result = _eval(ast.parse(expr, mode="eval").body)
        return str(round(result, 6)) if isinstance(result, float) else str(result)
    except ZeroDivisionError:
        return "Ошибка: деление на ноль"
    except Exception as e:
        return f"Ошибка: {e}"     # ошибку возвращаем агенту как Observation
```

Обратите внимание: инструмент **не выбрасывает исключение наружу**, а возвращает текст ошибки. Агент увидит его как Observation и сможет исправиться — это одно из ключевых свойств ReAct.

Поиск по тексту реализуете сами. Минимальные требования: прочитать все `.txt` из папки `docs/`, разбить на абзацы, оценить каждый абзац по пересечению слов с запросом и вернуть top-3 абзаца с указанием файла. Каркас:

Листинг 4. tools.py — каркас поиска (TODO выполняет студент)

```
import re
from pathlib import Path
DOCS_DIR = Path("docs")

def _tokenize(text: str) -> set[str]:
    return {w for w in re.findall(r"\w+", text.lower()) if len(w) > 2}

def text_search(query: str, top_k: int = 3) -> str:
    q = _tokenize(query)
    hits = []
    for path in sorted(DOCS_DIR.glob("*.txt")):
        for i, para in enumerate(path.read_text(encoding="utf-8").split("\n\n")):
            # TODO: вычислить score — число общих слов запроса и абзаца
            # TODO: если score > 0, добавить (score, path.name, i, para) в hits
            ...
    # TODO: если ничего не найдено — вернуть понятное сообщение
    # TODO: отсортировать hits по убыванию score, вернуть top_k строк
    #       в формате "[файл, абзац i] текст" (обрежьте текст до ~400 символов)
```

Вопрос для размышления

Слова «командировка», «командировки», «командировок» для такого поиска — разные токены. Как это влияет на агента? Попробуйте два решения: (а) сравнивать первые 5–6 букв слова (грубый стемминг); (б) позволить агенту переформулировать запрос, если поиск ничего не нашёл. Какое лучше и почему?

6. Шаг 2. Системный промпт

Промпт задаёт формат, перечисляет инструменты и правила. Он — главный «интерфейс» между вашим кодом и моделью: парсер сможет разобрать только то, что модель напишет в оговорённом формате.

Листинг 5. agent.py — системный промпт

```
SYSTEM_PROMPT = """Ты — ассистент, который решает задачи пошагово в формате ReAct.
Доступные инструменты:
{tools}

Строго соблюдай формат. На каждом шаге пиши:
Thought: рассуждение о том, что делать дальше
Action: имя_инструмента
Action Input: входные данные инструмента одной строкой

После этого ОСТАНОВИСЬ и жди строку Observation с результатом.
Когда информации достаточно, напиши:
Thought: итоговое рассуждение
```

```
Final Answer: ответ пользователю

Правила: не придумывай Observation сам; используй только перечисленные инструменты;
все вычисления делай через calculator; факты о компании бери только из text_search."""
```

7. Шаг 3. Парсер и цикл агента

Парсер определяет, что написала модель: финальный ответ, вызов инструмента или нечто, не соответствующее формату. Регулярные выражения даны; изучите, почему проверяется порядок совпадений.

Листинг 6. agent.py — парсер ответа модели

```
import re
ACTION_RE = re.compile(r"Action:\s*([\w\-]+)\s*\n\s*Action Input:\s*(.+)")
FINAL_RE  = re.compile(r"Final Answer:\s*(.+)", re.S)

def parse(text: str):
    """('final', answer) | ('action', name, arg) | ('error', message)"""
    text = text.split("Observation:")[0]        # модель могла «дописать» наблюдение
    m_act, m_fin = ACTION_RE.search(text), FINAL_RE.search(text)
    if m_fin and (not m_act or m_fin.start() < m_act.start()):
        return ("final", m_fin.group(1).strip())
    if m_act:
        return ("action", m_act.group(1).strip(), m_act.group(2).strip())
    return ("error", "Нет 'Action:'+'Action Input:' и нет 'Final Answer:'. Соблюдай формат.")
```

Теперь главное — цикл. Ниже каркас: логику внутри цикла пишете сами по схеме из раздела 2. Параметр `stop=["Observation:"]` просит API прекратить генерацию, как только модель начнёт сама «выдумывать» наблюдение.

Листинг 7. agent.py — каркас цикла ReAct (TODO выполняет студент)

```
import os
from openai import OpenAI
client = OpenAI(base_url=os.getenv("LLM_BASE_URL", "https://api.groq.com/openai/v1"),
                api_key=os.getenv("GROQ_API_KEY"))
MODEL = os.getenv("LLM_MODEL", "openai/gpt-oss-20b")
TOOLS = {"calculator": calculator, "text_search": text_search}

def run_agent(question: str, max_steps: int = 8) -> str:
    tools_desc = "- calculator: вычисляет арифметическое выражение\n" \
                 "- text_search: ищет фрагменты во внутренних документах"
    messages = [{"role": "system", "content": SYSTEM_PROMPT.format(tools=tools_desc)},
                {"role": "user", "content": f"Question: {question}"}]

    for step in range(1, max_steps + 1):
        resp = client.chat.completions.create(model=MODEL, messages=messages,
                                              temperature=0, stop=["Observation:"])
        text = (resp.choices[0].message.content or "").strip()
        parsed = parse(text)

        # TODO 1: если parsed[0] == "final" — записать шаг в журнал и вернуть ответ
        # TODO 2: если "action" — найти инструмент в TOOLS и выполнить его;
        #         неизвестный инструмент -> observation с текстом ошибки и списком TOOLS
        # TODO 3: если "error" — observation = сообщение о нарушении формата
        # TODO 4: записать шаг в журнал (см. листинг 8)
        # TODO 5: добавить в messages ответ модели (role="assistant")
        #         и наблюдение (role="user", content=f"Observation: {observation}")

    return "Не удалось получить ответ за отведённое число шагов."
```

Подсказки, если застряли

Наблюдение передаётся сообщением с ролью `user`, потому что в текстовом ReAct модель «не знает» о роли tool — для неё это просто очередная реплика среды.

Если провайдер возвращает ошибку на параметр `stop`, уберите его: парсер всё равно обрежет текст по «Observation:».

Проверяйте цикл без сети: подставьте вместо вызова API функцию, которая по очереди возвращает заранее написанные ответы (mock). Так отлаживаются парсер и логирование.

8. Шаг 4. Журнал шагов

Журнал — обязательная часть задания. Используйте формат **JSON Lines** (одна JSON-запись на строку): его легко дописывать, читать pandas и загружать в LangSmith или Langfuse на неделе 8. Каждая запись должна содержать как минимум поля из примера:

Листинг 8. Пример записи журнала (agent_log.jsonl)

```
{"ts": "2026-10-12T09:14:03Z", "run_id": "20261012-091402", "step": 1, "event": "step",
 "llm_output": "Thought: Нужна ставка суточных...\nAction: text_search\nAction Input: суточные",
 "action": "text_search", "action_input": "суточные", "observation": "[hr_policy.txt, абзац 1] ...",
 "tool_ok": true, "latency_s": 0.84, "usage": {"prompt_tokens": 412, "completion_tokens": 58}}
```

Подсказки: `json.dumps(record, ensure_ascii=False)` сохранит кириллицу читаемой; время ответа модели измеряйте `time.perf_counter()`; число токенов есть в `resp.usage`. Записывайте отдельные события `start`, `step`, `final` и `max_steps_reached`.

9. Шаг 5. Тестирование и отладка

9.1. Отладка без сети: подставная модель

Парсер, вызов инструментов и журнал удобно проверять без обращения к API: это быстро, бесплатно и воспроизводимо. Для этого вынесите вызов модели в отдельную функцию `call_llm(messages) -> str` и передавайте её в `run_agent` параметром. Тогда в тестах её можно заменить «подставной» моделью, которая возвращает заранее написанные ответы.

Листинг 9. Подставная модель для проверки цикла

```
def make_mock_llm(script):
    """Возвращает заранее написанные ответы по очереди — отладка без API."""
    replies = iter(script)
    return lambda messages: next(replies)

mock = make_mock_llm([
    "Thought: Нужна ставка суточных.\nAction: text_search\nAction Input: суточные",
    "Thought: Попробую несуществующий инструмент.\nAction: weather\nAction Input: Бишкек",
    "Thought: Посчитаю.\nAction: calculator\nAction Input: 1500 * 4",
    "Thought: Готово.\nFinal Answer: 6000 сом",
])
# run_agent("Сколько суточных за 4 дня?", llm=mock)
# Ожидание: 4 шага, на 2-м — Observation с ошибкой «неизвестный инструмент»
```

Добавьте в сценарий ответ без `Action` и без `Final Answer` и убедитесь, что агент получает Observation с сообщением о нарушении формата, а не падает с исключением.

9.2. Тестовые вопросы

Подготовьте 5 вопросов и **заранее** запишите правильные ответы — иначе оценка превращается в «на глаз». Используйте таблицу как шаблон для отчёта:

№	Тип вопроса	Вопрос	Ожидаемый ответ	Ответ агента	Шаги	Токены	Верно?
1	Поиск						

№	Тип вопроса	Вопрос	Ожидаемый ответ	Ответ агента	Шаги	Токены	Верно?
2	Поиск						
3	Вычисление						
4	Поиск + вычисление						
5	Поиск + вычисление						

9.3. Метрики по журналу

После прогона тестов посчитайте метрики по `agent_log.jsonl`. Удобно использовать pandas: формат JSON Lines читается одной строкой.

Листинг 10. Начало анализа журнала

```
import pandas as pd
df = pd.read_json("agent_log.jsonl", lines=True)

steps = df[df.event == "step"].groupby("run_id").size()
print("Среднее число шагов на вопрос:", steps.mean())
print("Средняя задержка LLM, с:", df["latency_s"].mean())
# TODO: доля успешных вызовов инструментов (колонка tool_ok)
# TODO: суммарное число токенов (поле total_tokens внутри колонки usage)
```

10. Задания

Базовая часть (обязательно)

1. Реализовать `text_search` и цикл `run_agent` по каркасам из листингов 4 и 7; агент работает на Groq или Ollama.
2. Подготовить 3–5 документов в `docs/` и 5 тестовых вопросов: 2 — только поиск, 1 — только вычисление, 2 — поиск + вычисление.
3. Прогнать тесты, сохранить журнал и посчитать: долю правильных ответов, среднее число шагов, среднюю задержку, суммарное число токенов.
4. Намеренно «сломать» агента: задать вопрос, ответа на который нет в документах, и вопрос, провоцирующий вызов несуществующего инструмента. Описать по журналу поведение агента.

Продвинутая часть (одно задание на выбор, бонус)

- **A. Нативный function calling.** Переписать агента на параметр `tools` API (листинг 5 учебного материала недели 4) и сравнить с текстовой версией на тех же 5 вопросах: точность, шаги, токены, ошибки формата.
- **B. Сравнение моделей.** Прогнать тесты на двух моделях (например, `openai/gpt-oss-20b` и `openai/gpt-oss-120b` или облачная модель и локальная через Ollama) — это заготовка раздела оценки итогового проекта.
- **C. Простой Reflexion.** Если ответ не прошёл проверку (например, нет числа, когда оно ожидалось), агент пишет рефлексию и делает вторую попытку с ней в контексте.
- **D. Human-in-the-loop.** Добавить инструмент `ask_human(question)`, который запрашивает уточнение у пользователя через `input()`.

11. Отчёт и критерии оценивания

Отчёт сдаётся до следующего занятия в LMS. Отчёты по лабораторным составляют 20% итоговой оценки курса; сдача в течение 48 часов после дедлайна — со скидкой 20%.

Раздел отчёта	Содержание
Что сделал	Схема цикла, ключевые решения, ссылка на репозиторий или архив с кодом
Результаты	Заполненная таблица из раздела 9.2 и метрики из раздела 9.3

Раздел отчёта	Содержание
Что не получилось	1–2 ошибки агента с фрагментом журнала и гипотезой о причине
Что усвоил	3–5 предложений своими словами
Использование ИИ	Какие ИИ-инструменты применялись и как (обязательное правило курса)

Критерий	Баллы
Цикл ReAct работает, корректно обрабатывает final / action / ошибку формата / неизвестный инструмент	30
Инструмент поиска реализован и возвращает источник фрагмента	15
Журнал JSONL содержит все требуемые поля	20
5 тестов с метриками и анализ двух сценариев «поломки»	25
Качество отчёта и кода (комментарии, понятные имена)	10
Продвинутое задание (бонус)	+10

12. Типичные ошибки и отладка

Симптом	Вероятная причина	Что сделать
Модель сама пишет «Observation: ...» с выдуманным результатом	Не задан stop или модель игнорирует инструкцию	Параметр stop + обрезка текста по «Observation:» в парсере
Агент зацикливается, повторяя один и тот же поиск	Поиск ничего не находит, а агент не меняет стратегию	Информативное сообщение «ничего не найдено, переформулируй»; лимит max_steps
Ошибка формата на каждом шаге	Модель слабо следует инструкциям или промпт неоднозначен	Добавить пример траектории в промпт (few-shot), снизить temperature, сменить модель
Агент считает «в уме» и ошибается	Нет правила использовать калькулятор	Явное правило в промпте и в описании инструмента
Ошибка model_not_found / decommissioned	Модель снята с обслуживания	Проверить список моделей (листинг 1) и страницу deprecations [9]

13. Контрольные вопросы

Вопросы охватывают теорию недели 4 и практику лабораторной работы. Ответы приведены в разделе 14.

1. Чем агент отличается от workflow по определению Anthropic? Когда агент не нужен?
2. Из каких компонентов состоит LLM-агент по формуле Лилиан Венг?
3. Из каких элементов состоит шаг ReAct и кто формирует каждый из них?
4. Какие проблемы Chain-of-Thought решает ReAct и какие результаты показала оригинальная статья?
5. Зачем в лабораторной используется параметр stop=["Observation:"] и обрезка ответа модели по этой строке?
6. Зачем ограничивать число шагов агента (max_steps) и что должен делать агент при исчерпании лимита?
7. Как устроен паттерн Plan-and-Execute и в каких задачах он выгоднее ReAct?
8. Из каких компонентов состоит Reflexion и при каком условии его имеет смысл применять?
9. Сравните in-context, векторную и структурированную (БД) память агента.
10. Что такое эффект «Lost in the Middle» и что он означает для проектирования памяти агента?
11. Опишите механику нативного function calling. Кто выполняет функцию и от чего зависит выбор инструмента?
12. Какие значения параметра tool_choice есть в Anthropic API и как принудительный вызов помогает извлекать структурированные данные?

13. Чем текстовый ReAct из лабораторной отличается от нативного function calling? Почему в работе выбран текстовый вариант?
14. Почему калькулятор на eval() опасен и как защищён калькулятор в лабораторной? Какой риск OWASP с этим связан?
15. Почему инструменты агента возвращают ошибку текстом, а не выбрасывают исключение, и зачем каждая такая ошибка записывается в журнал?

14. Ответы на контрольные вопросы

1. Чем агент отличается от workflow по определению Anthropic? Когда агент не нужен?

В workflow LLM и инструменты связаны заранее заданными путями в коде: последовательность шагов определяет разработчик. В агенте LLM сама динамически решает, какие шаги сделать, какие инструменты вызвать и когда остановиться. Anthropic рекомендует начинать с самого простого решения: если задачу решает один вызов LLM или фиксированный workflow, агент не нужен — он дороже, менее предсказуем и может накапливать ошибки.

2. Из каких компонентов состоит LLM-агент по формуле Лилиан Венг?

Агент = LLM + планирование + память + использование инструментов. LLM выступает «мозгом», планирование отвечает за разбиение задачи на шаги и самокритику, память — за сохранение контекста и опыта, инструменты — за доступ к информации и действиям за пределами знаний модели.

3. Из каких элементов состоит шаг ReAct и кто формирует каждый из них?

Thought (рассуждение) и Action с Action Input (имя инструмента и входные данные) пишет модель. Observation (результат инструмента) формирует программный код после реального выполнения инструмента. Final Answer пишет модель, когда информации достаточно; он завершает цикл.

4. Какие проблемы Chain-of-Thought решает ReAct и какие результаты показала оригинальная статья?

Chain-of-Thought опирается только на знания внутри модели, что приводит к галлюцинациям и распространению ошибок по цепочке рассуждений. ReAct получает факты из внешних источников через действия. По данным Yao et al. (ICLR 2023), на бенчмарках ALFWorld и WebShop ReAct превзошёл методы имитационного обучения и обучения с подкреплением на 34 и 10 процентных пунктов успешности, используя один-два примера в промпте.

5. Зачем в лабораторной используется параметр stop=["Observation:"] и обрезка ответа модели по этой строке?

Модель, обученная на текстах с примерами ReAct, может продолжить генерацию и сама «придумать» Observation — выдуманный результат инструмента. Если такой текст попадёт в историю, агент будет рассуждать на основе галлюцинации. Параметр stop просит API остановить генерацию на этой строке, а обрезка в парсере страхует на случай, если провайдер или модель параметр не поддерживает.

6. Зачем ограничивать число шагов агента (max_steps) и что должен делать агент при исчерпании лимита?

Без лимита агент может зациклиться — например, повторять один и тот же безуспешный поиск, — расходуя время и деньги на каждый вызов LLM. Лимит гарантирует завершение работы и предсказуемый бюджет. При исчерпании лимита агент должен вернуть понятное сообщение о том, что ответ не получен, и записать это событие в журнал.

7. Как устроен паттерн Plan-and-Execute и в каких задачах он выгоднее ReAct?

Планировщик (LLM) составляет полный план из шагов, исполнитель выполняет шаги по одному (исполнителем может быть ReAct-агент), а на этапе перепланирования проверяется прогресс и при необходимости корректируются оставшиеся шаги. Паттерн выгоднее для длинных многоэтапных задач с понятной структурой: сильная модель нужна только для плана, а независимые шаги можно выполнять параллельно (ReWOO, LLMCompiler). Основа подхода — Plan-and-Solve Prompting (Wang et al., ACL 2023). Главный риск — план устаревает, если среда меняется по ходу работы.

8. Из каких компонентов состоит Reflexion и при каком условии его имеет смысл применять?

Исполнитель (Actor) делает попытку, оценщик (Evaluator) проверяет результат, модуль саморефлексии превращает сигнал оценщика в текстовый вывод о причинах неудачи, который сохраняется в эпизодическую память и учитывается в следующей попытке. По данным Shinn et al. (NeurIPS 2023), так был достигнут 91% pass@1 на HumanEval против 80% у GPT-4. Применять подход стоит, когда результат можно надёжно проверить автоматически (unit-тесты, выполнение SQL, валидация схемы) и качество важнее стоимости: несколько попыток кратно увеличивают расходы.

9. Сравните in-context, векторную и структурированную (БД) память агента.

In-context память — информация прямо в запросе к модели (история диалога и шагов); передаётся целиком, ограничена размером контекстного окна и увеличивает стоимость каждого вызова. Векторная память хранит эмбеддинги текстов в векторной БД (ChromaDB, Qdrant, pgvector) и извлекается

семантическим поиском top-k; риск — нерелевантные или устаревшие фрагменты. Структурированная память в обычной БД (PostgreSQL, MongoDB, Redis) хранит точные факты — профили, статусы, транзакции — и доступна через инструменты; ключевые риски — права доступа и персональные данные.

10. Что такое эффект «Lost in the Middle» и что он означает для проектирования памяти агента?

Исследование Liu et al. (2023) показало, что качество ответов модели обычно выше, когда нужная информация находится в начале или в конце контекста, и заметно падает, если она в середине. Поэтому «положить всё в контекст» — плохая стратегия памяти: лучше передавать модели минимально достаточный набор данных, обрезать и суммаризировать старую историю, сжимать результаты инструментов и размещать важное ближе к началу или концу.

11. Опишите механику нативного function calling. Кто выполняет функцию и от чего зависит выбор инструмента?

Разработчик передаёт в запросе список инструментов с именем, описанием и JSON Schema параметров. Модель отвечает либо текстом, либо структурированным запросом вызова (имя + аргументы в JSON). Функцию выполняет код приложения, а результат возвращается модели следующим сообщением. Модели специально обучены этому навыку; на выбор сильнее всего влияют понятное имя, подробное описание (когда использовать и когда нет), схема параметров, правила в системном промпте и небольшое число хорошо разграниченных инструментов.

12. Какие значения параметра tool_choice есть в Anthropic API и как принудительный вызов помогает извлекать структурированные данные?

auto — модель сама решает, вызывать ли инструмент (по умолчанию); any — обязана вызвать один из инструментов; tool — обязана вызвать указанный инструмент; none — вызовы запрещены. В OpenAI-совместимых API аналогичные значения: auto, required, none или конкретная функция. Если определить один инструмент, схема которого совпадает с нужной структурой (например, накладная), и заставить модель его вызвать, она гарантированно вернёт данные в этой структуре.

13. Чем текстовый ReAct из лабораторной отличается от нативного function calling? Почему в работе выбран текстовый вариант?

В текстовом ReAct модель пишет вызов свободным текстом в оговорённом формате, а код разбирает его регулярными выражениями; это работает с любой instruct-моделью, но формат может быть нарушен. В нативном function calling вызов приходит структурированным JSON по схеме, разбор надёжнее, поддерживаются параллельные вызовы, но механика скрыта в API. Текстовый вариант выбран из учебных соображений: в нём виден весь механизм агента — промпт, парсинг, цикл и обработка ошибок.

14. Почему калькулятор на eval() опасен и как защищён калькулятор в лабораторной? Какой риск OWASP с этим связан?

Строку для eval() формирует LLM, а значит в неё может попасть произвольный код, например вызов `__import__("os").system(...)`, в том числе через prompt injection из найденных документов. Калькулятор в лабораторной разбирает выражение модулем ast и вычисляет только разрешённые узлы — числа и арифметические операторы, — а также ограничивает степень, чтобы нельзя было вызвать огромное вычисление. Чрезмерные функции, права или автономность агента описаны в OWASP Top 10 for LLM Applications 2025 как риск LLM06: Excessive Agency.

15. Почему инструменты агента возвращают ошибку текстом, а не выбрасывают исключение, и зачем каждая такая ошибка записывается в журнал?

Ошибка, возвращённая как текст, становится Observation: модель видит, что пошло не так, и может исправиться — переформулировать запрос, исправить выражение или выбрать другой инструмент. Исключение просто остановило бы программу. Запись в журнал JSONL (действие, вход, наблюдение, признак tool_ok, задержка, токены) позволяет считать метрики качества, находить систематические ошибки, расследовать инциденты, а на неделе 8 — загрузить трассировки в LangSmith или Langfuse для оценки агента.

15. Ссылки

[1] Yao S. et al. ReAct: Synergizing Reasoning and Acting in Language Models. ICLR 2023. https://arxiv.org/abs/2210.03629

[2] Shinn N. et al. Reflexion: Language Agents with Verbal Reinforcement Learning. NeurIPS 2023. https://arxiv.org/abs/2303.11366

[3] Wang L. et al. Plan-and-Solve Prompting. ACL 2023. https://aclanthology.org/2023.acl-long.147

[4] Schluntz E., Zhang B. (Anthropic). Building Effective Agents. 2024. https://www.anthropic.com/engineering/building-effective-agents

[5] Liu N. F. et al. Lost in the Middle: How Language Models Use Long Contexts. TACL 2024. https://arxiv.org/abs/2307.03172

[6] Anthropic. Tool use — Define tools (документация). https://platform.claude.com/docs/en/agents-and-tools/tool-use/define-tools

[7] OWASP GenAI Security Project. Top 10 for LLM Applications 2025. https://genai.owasp.org/llm-top-10/

[8] Groq. Supported Models (документация GroqCloud). https://console.groq.com/docs/models

[9] Groq. Model Deprecations. https://console.groq.com/docs/deprecations

[10] Ollama. OpenAI compatibility. https://ollama.com/blog/openai-compatibility